

Analisi di Sicurezza APK

Enpass Password Manager 5.6.9

SICUREZZA IN AMBIENTI MOBILE

CANDIDATO:

Matteo Esposito
806075

DOCENTE:

Prof. Paolo Buono

Enpass Password Manager

Gestore di password che custodisce credenziali e dati sensibili in un vault cifrato, sbloccabile solo con la Master Password.

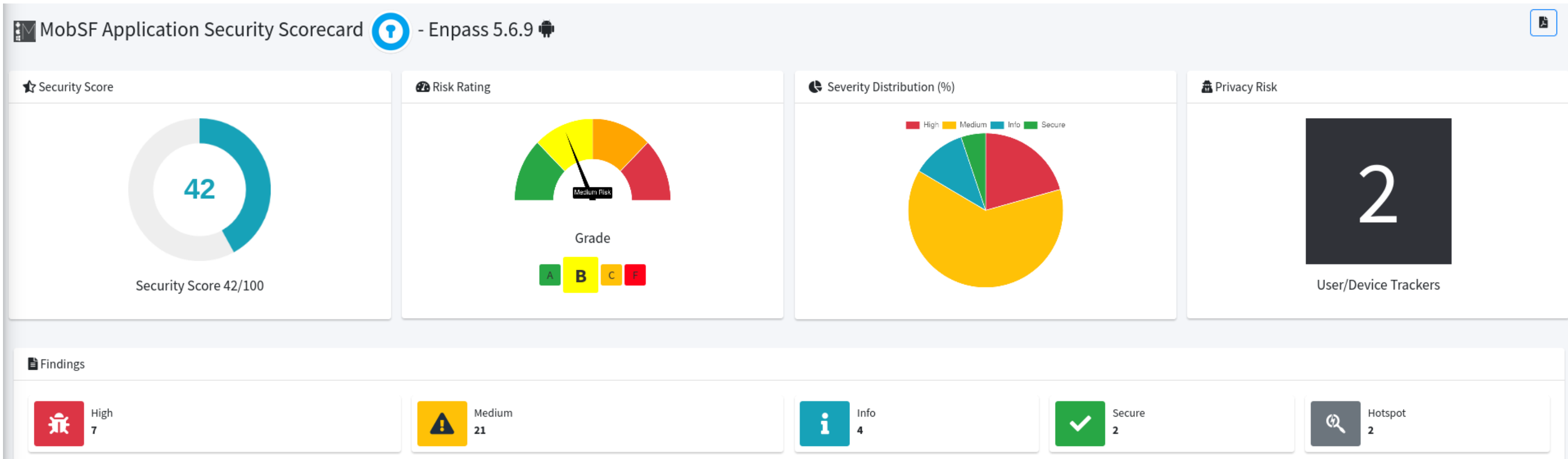
- Database cifrato a livello di file con SQLCipher e HMAC per pagina, illeggibile a riposo senza la Master Password.
- Sblocco locale tramite Master Password, PIN o impronta, con chiave biometrica ancorata all'Android KeyStore.
- Compilazione automatica delle credenziali e browser integrato isolato per la navigazione sensibile.
- Sincronizzazione del vault cifrato su account cloud dell'utente.
- Generatore di Password complesse.



Strumenti Utilizzati

Strumento	Scopo
Kali Linux	Sistema operativo di base per il penetration testing mobile
AVD	Emulatore per testare l'app in un ambiente virtuale e isolato
MobSF	Framework per l'analisi di sicurezza automatizzata dell'APK
jadx	Decompilatore per estrarre e leggere il codice sorgente dell'app
Frida	Strumento per manipolare e analizzare l'app in tempo reale mentre è in esecuzione
objection	Console interattiva che semplifica l'utilizzo di Frida tramite comandi pronti all'uso
adb	Interfaccia a riga di comando per far comunicare il computer con l'emulatore
mitmproxy	Proxy per intercettare e ispezionare il traffico di rete HTTP/HTTPS
apksigner	Strumento per verificare la validità della firma digitale del pacchetto APK

Analisi con MobSF



APP SCORES

Security Score

42/100

Trackers Detection

2/432

MobSF Scorecard

FILE INFORMATION

File Name

enpass-password-manager-5-6-9.apk

Size

20.98MB

MDS

c0e2fcea782778b02a22b604f81ee80f

SHA1

22ff51783a6f8c9e1069a2cff905f85af8086ee1

SHA256

2ea637f6444c9277f14121ffcacc1d2e7084a2e6d97eafc016e625a0c2e33607

APP INFORMATION

App Name

Enpass

Package Name

io.enpass.app

Main Activity

in.sinew.enpass.MainActivity

Target SDK

24

Min SDK

15

Max SDK

Android Version Name

5.6.9

Android Version Code

136

13 Vulnerabilità

#	Vulnerabilità	MASVS v2
1	Assenza di device-access-security policy	STORAGE
2	Esposizione Master Password in memoria	STORAGE
3	Mancato aggiornamento Security Provider	NETWORK
4	Assenza di certificate pinning	NETWORK
5	RCE via addJavascriptInterface	PLATFORM
6	Esposizione a overlay e tapjacking	PLATFORM
7	Assenza di aggiornamento obbligatorio	CODE
8	Inefficacia dei meccanismi di root detection	RESILIENCE
9	Assenza di emulator detection	RESILIENCE
10	Assenza di controlli di integrità del codice	RESILIENCE
11	Assenza rilevamento instrumentazione dinamica	RESILIENCE
12	Assenza di offuscamento del codice	RESILIENCE
13	Assenza di rilevamento anti-debugging	RESILIENCE

Assenza di una device-access-security policy

MASVS-STORAGE

Descrizione

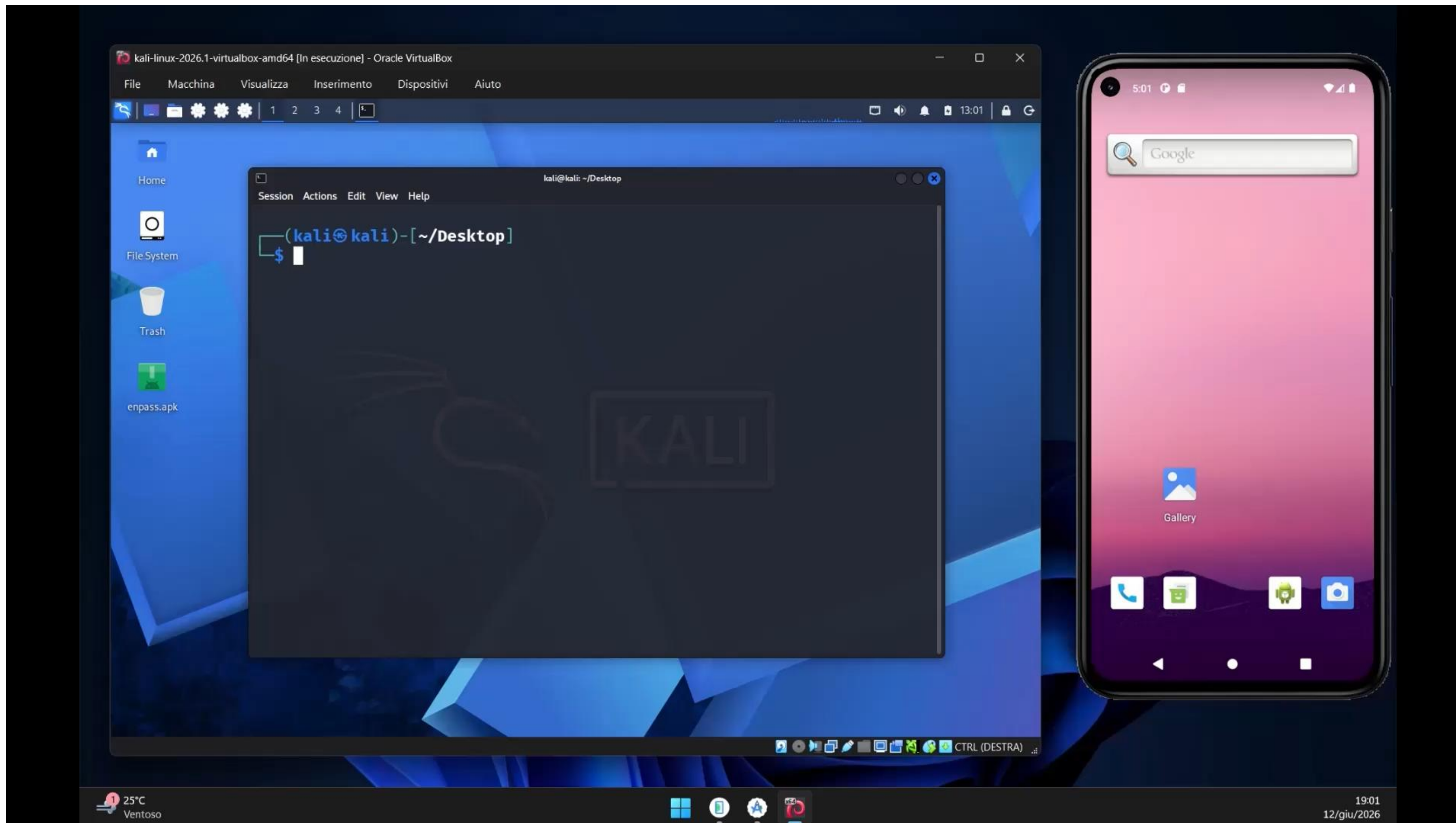
- Nessun requisito minimo sulla sicurezza del dispositivo imposto prima di consentire l'accesso al vault.
- Nessun uso della Device Administration API, unico controllo `isKeyguardSecure()` come pre-condizione opzionale dello sblocco biometrico.
- Confermata su emulatore non conforme (senza blocco schermo, rooted, debuggable) l'app viene installata e vault usato senza restrizioni.

Mitigazione

- Introdurre una device-access-security policy esplicita che, sui dispositivi non conformi, avvisi l'utente o limiti le operazioni più sensibili.

Assenza di una device-access-security policy

MASVS-STORAGE



Esposizione della Master Password nella memoria

MASVS-STORAGE

Descrizione

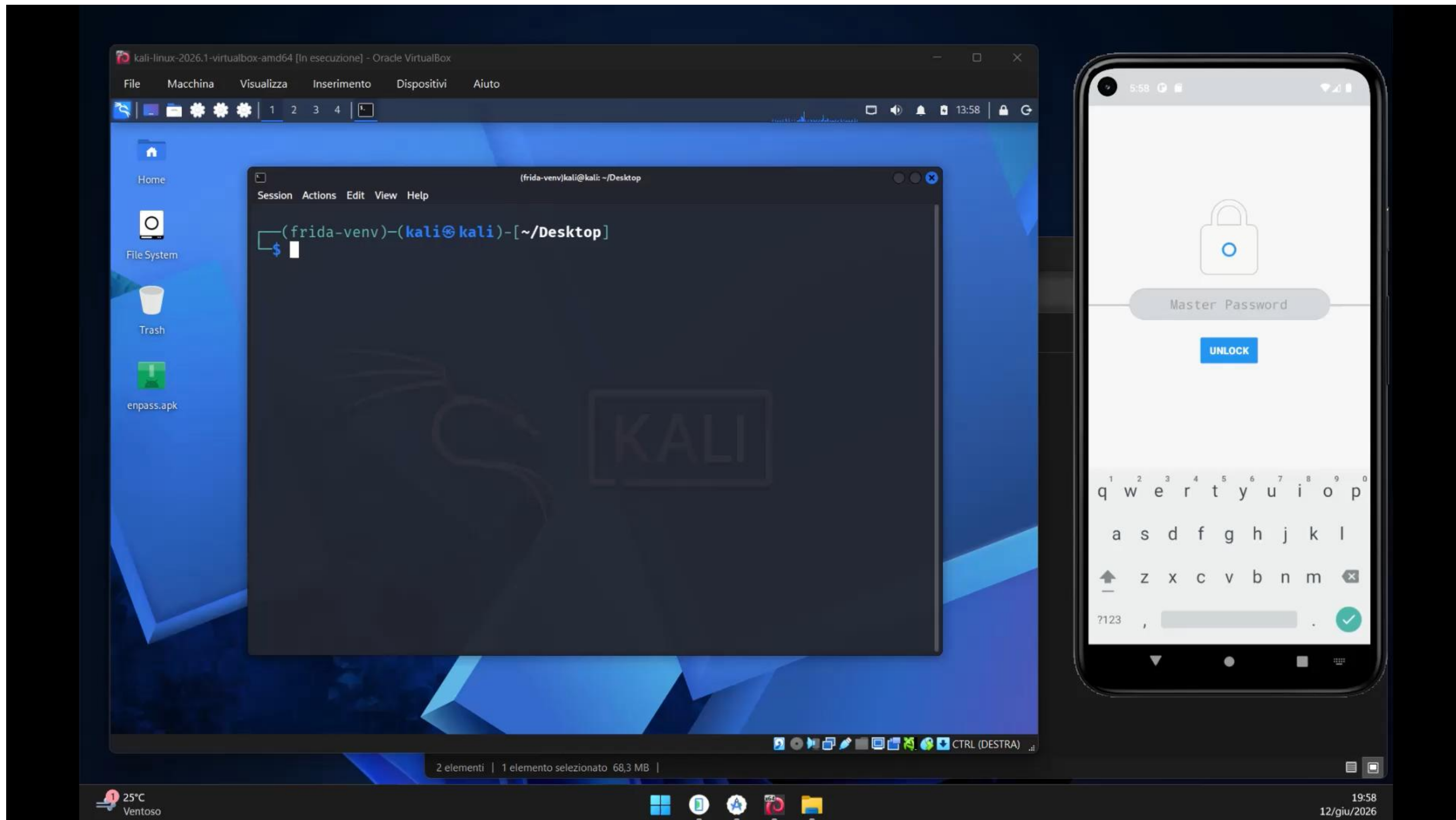
- Uso estensivo di tipi immutabili come String, la password viene convertita e copiata.
- Con objection e app sbloccata, memory search ha trovato la Master Password in chiaro e in più copie nella memoria di processo.
- Recuperati anche dati del vault decifrato, persistenza ben oltre l'istante dello sblocco.

Mitigazione

- Gestire i segreti solo con `char[]/byte[]`, sovrascriverli dopo l'uso, evitare le conversioni in String e adottare una `Editable.Factory` sicura per il campo password.

Esposizione della Master Password nella memoria

MASVS-STORAGE



Mancato aggiornamento Security Provider

MASVS-NETWORK

Descrizione

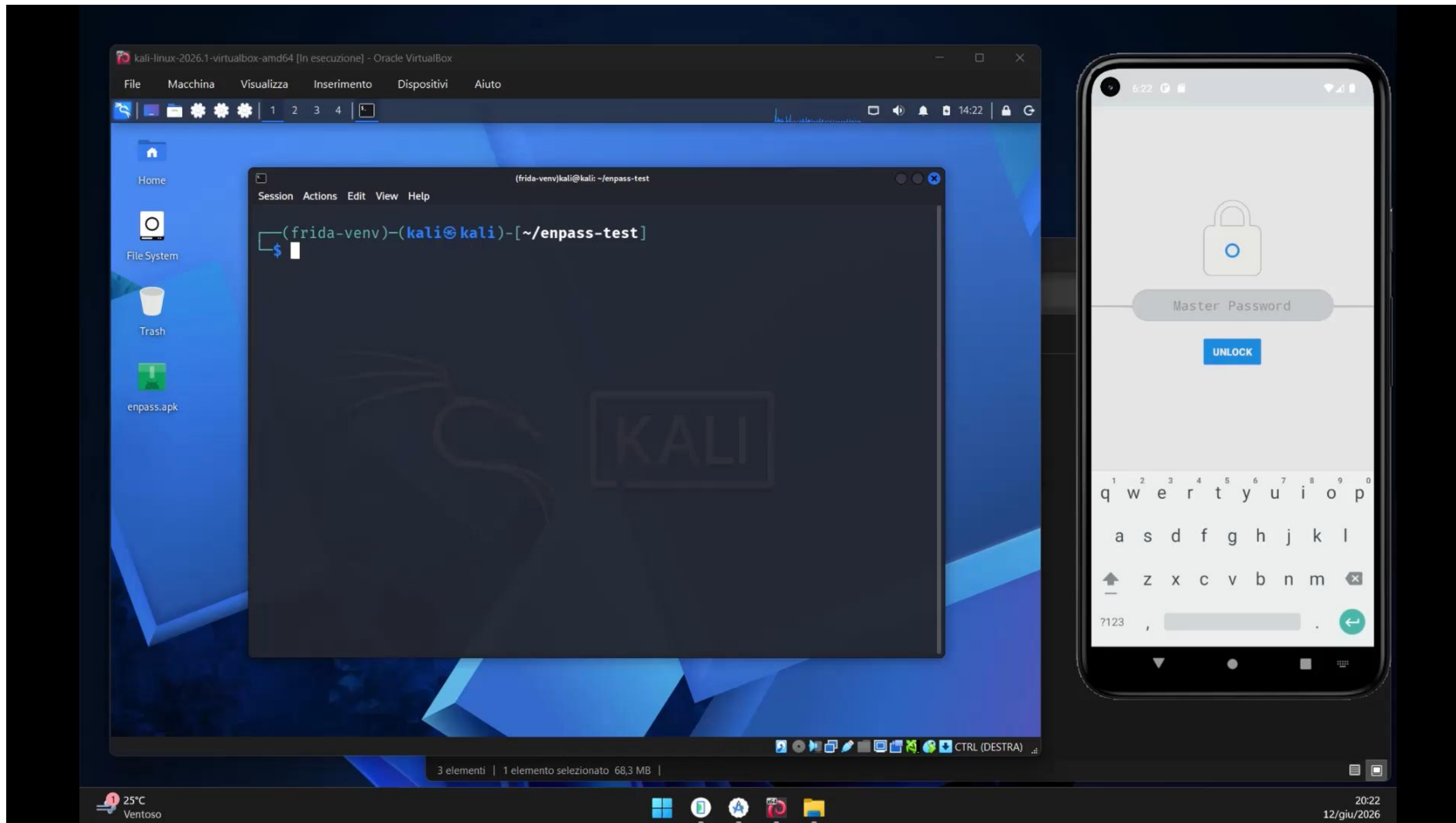
- L'app non invoca i metodi di aggiornamento dei Google Play Services (ProviderInstaller.installIfNeeded), affidandosi al provider crittografico di base del sistema.
- Con un minSdkVersion pari a 15, l'esecuzione su dispositivi Android datati lascia il provider obsoleto.
- L'analisi dinamica conferma che il provider aggiornato non viene caricato, esponendo il traffico a vulnerabilità SSL/TLS note non patchate.

Mitigazione

- Invocare esplicitamente l'aggiornamento del security provider il prima possibile nel ciclo di vita dell'applicazione, gestendo le relative eccezioni.

Mancato aggiornamento Security Provider

MASVS-NETWORK



Assenza di certificate pinning

Descrizione

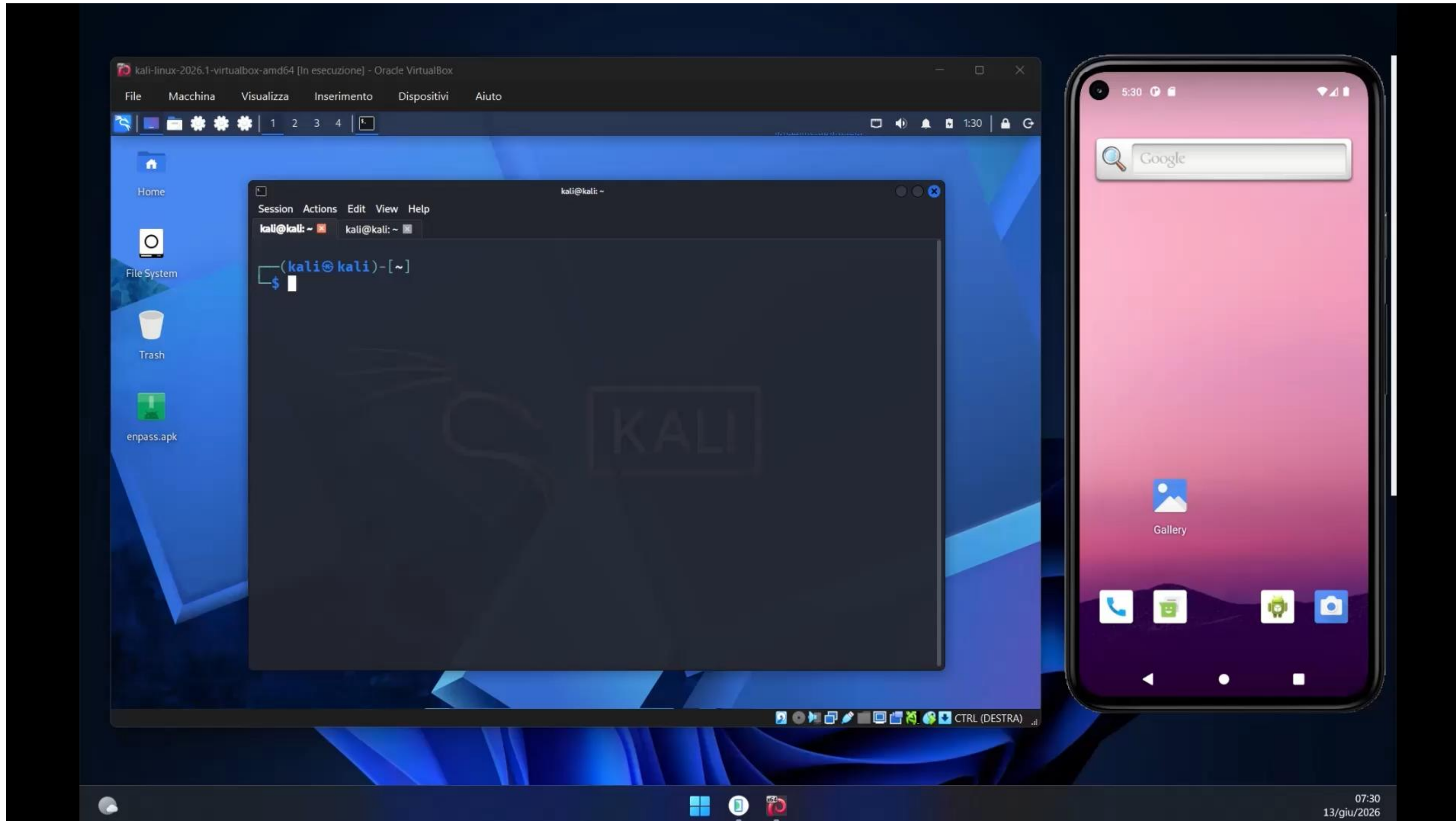
- L'applicazione non implementa alcuna forma di certificate pinning.
- Si fida in modo indiscriminato dell'intero store delle Certificate Authority del sistema operativo.
- Confermata su dispositivo, inserendo una CA fidata di sistema tramite proxy, il traffico TLS risulta intercettabile e ispezionabile in chiaro.

Mitigazione

- Implementare il certificate pinning per le destinazioni di rete controllate dallo sviluppatore, includendo pin di backup per evitare disservizi.

Assenza di certificate pinning

MASVS-NETWORK



RCE via addJavaScriptInterface

MASVS-PLATFORM

Descrizione

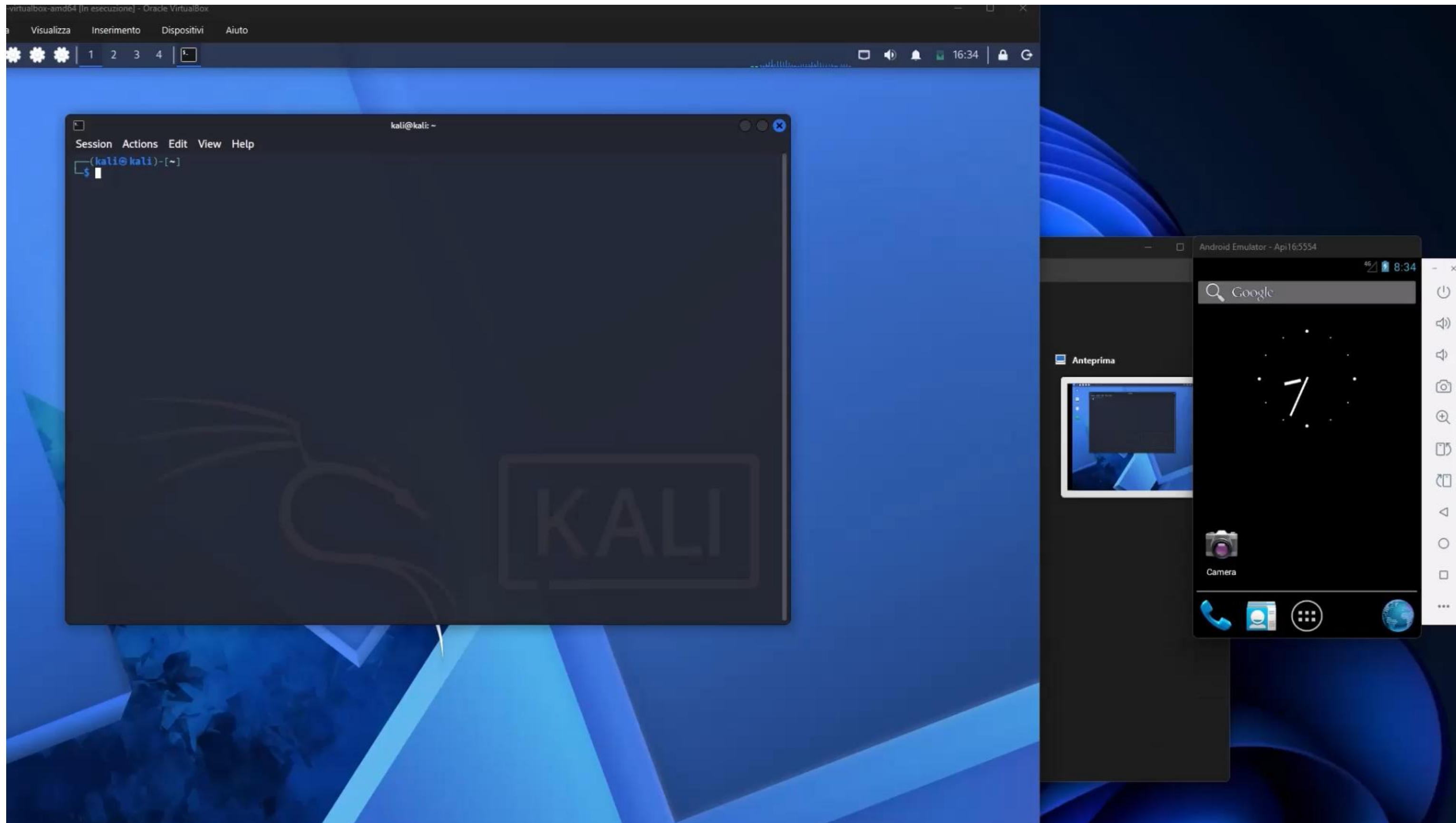
- Il browser interno espone un oggetto Java al codice JavaScript all'interno di una WebView che carica contenuti remoti.
- Essendo il minSdkVersion pari a 15, sui dispositivi con Android < 4.2 la reflection permette di raggiungere comandi di sistema sensibili.
- Testato con successo su emulatore 4.1.2 un payload JavaScript ha eseguito codice arbitrario ottenendo i privilegi dell'applicazione e accesso alla sandbox.

Mitigazione

- Elevare il minSdkVersion ad almeno 17 o limitare rigorosamente il caricamento di contenuti tramite il bridge solo a file locali attendibili e su canali HTTPS forzati.

RCE via addJavaScriptInterface

MASVS-PLATFORM



Esposizione a overlay e tapjacking

MASVS-PLATFORM

Descrizione

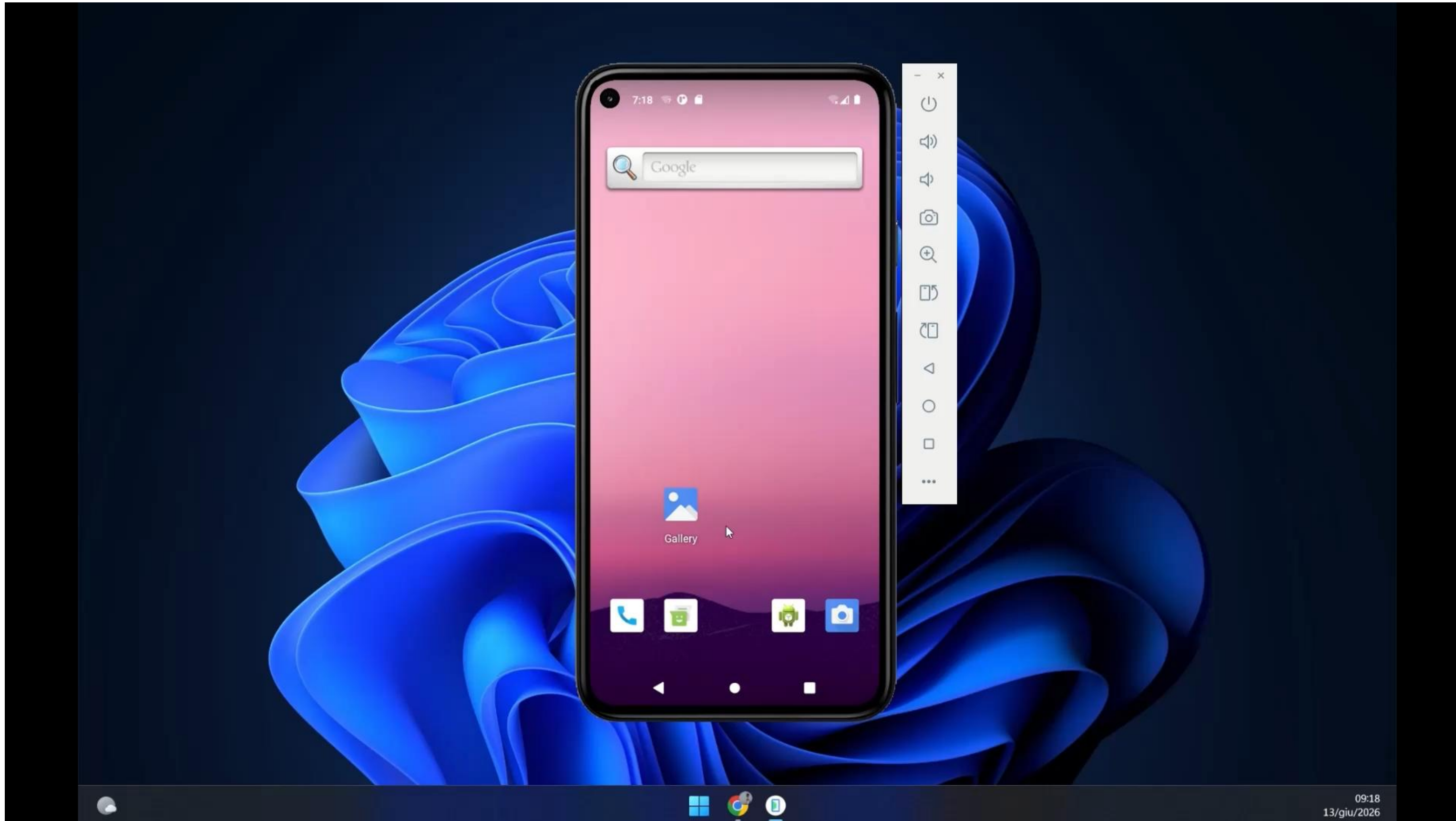
- Nessuna contromisura di touch filtering (es. `filterTouchesWhenObscured`) è presente sulle schermate sensibili (inserimento Master Password, PIN, conferme).
- Il `targetSdkVersion` pari a 24 esclude l'app dall'hardening di piattaforma contro le finestre in sovrapposizione introdotto in Android 8.
- Verificato tramite Proof-of-Concept: un'app malevola con permessi di disegno su schermo riesce a sovrapporsi integralmente intercettando i tocchi (Tapjacking).

Mitigazione

- Applicare l'attributo `android:filterTouchesWhenObscured="true"` sulle viste che ricevono input sensibili e aggiornare il `targetSdkVersion`.

Esposizione a overlay e tapjacking

MASVS-PLATFORM



Assenza di aggiornamento obbligatorio

MASVS-CODE

Descrizione

- L'app non impiega le API ufficiali In-App Updates, ma usa un controllo proprietario che interroga un endpoint obsoleto.
- L'avviso di aggiornamento è puramente informativo: l'utente può annullarlo o disabilitare del tutto i controlli tramite le preferenze.
- L'assenza di blocchi permette di continuare a utilizzare indefinitamente versioni vecchie, lasciando l'utente esposto a vulnerabilità critiche note.

Mitigazione

- Implementare un aggiornamento forzato o imporre una versione minima supportata lato server.

Inefficacia dei meccanismi di Root Detection

Descrizione

- I controlli per rilevare il root sono standard, non originali e concentrati unicamente in una singola classe a livello Java.
- Il rilevamento si limita a innescare un avviso informativo a schermo che non blocca in alcun modo le funzionalità critiche.
- L'intera analisi del penetration test è stata svolta su emulatore rooted senza ostacoli: i controlli sono banalmente aggirabili con Frida o disattivando il popup.

Mitigazione

- Distribuire controlli robusti e renderli vincolanti, bloccando le operazioni sensibili in caso di violazione.

Assenza di emulator detection

Descrizione

- Non è presente nel codice alcuna tecnica per il rilevamento di un ambiente virtualizzato o emulato.
- Le interrogazioni alle proprietà hardware assolvono esclusivamente a funzioni applicative lecite e non a verifiche di sicurezza.
- L'esecuzione su emulatore è avvenuta senza che l'app emettesse alcun avviso, agevolando l'analisi dinamica e gli attacchi automatizzati.

Mitigazione

- Introdurre controlli incrociati su indicatori hardware e software vincolando l'esecuzione dell'app.

Assenza di controlli di integrità del codice

Descrizione

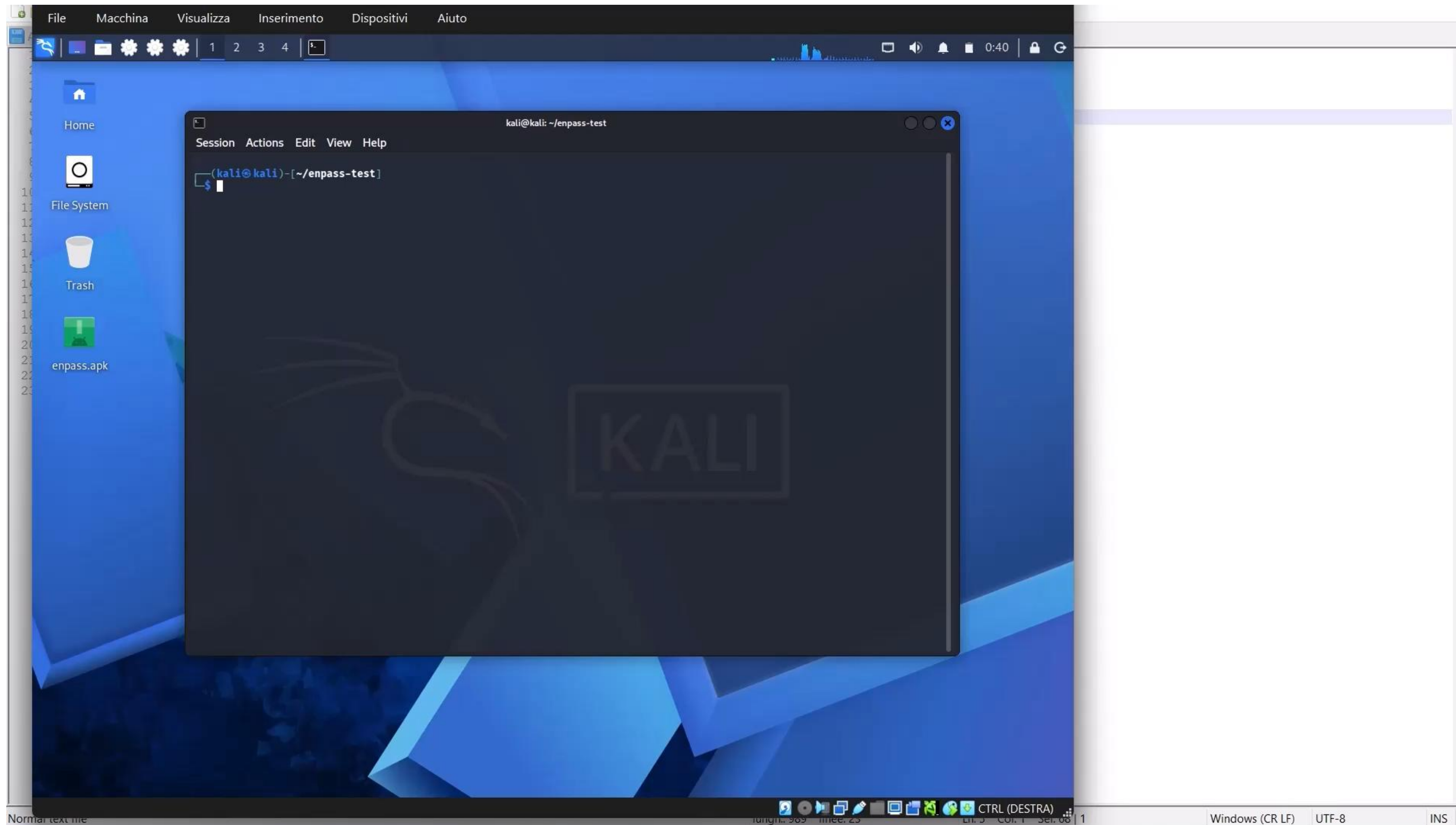
- L'app non verifica a runtime il proprio certificato di firma né calcola checksum per i file .dex o per le librerie native.
- Questa assenza consente a un attaccante di modificare l'APK, rifirmarlo e ridistribuirlo senza essere rilevato.

Mitigazione

- Inserire verifiche a runtime della firma del pacchetto originale e controlli di integrità sui file eseguibili e sulle librerie incluse.

Assenza di controlli di integrità del codice

MASVS-RESILIENCE



Assenza rilevamento instrumentation dinamica

Descrizione

- L'applicazione non cerca di individuare i noti framework di hooking a runtime come Frida, Xposed o Substrate.
- Mancano controlli su processi associati a questi strumenti, porte note o scansioni delle librerie caricate in `/proc/self/maps`.
- Un attaccante può agganciare liberamente i metodi Java e nativi a runtime per scavalcare verifiche biometriche o estrarre segreti, senza reazioni difensive.

Mitigazione

- Implementare ispezioni della memoria, rilevamento di named pipe, scansione porte e tecniche di anti-hooking nativo per difendersi dall'analisi a runtime.

Assenza di offuscamento del codice

Descrizione

- L'APK decompilato presenta codice di prima parte leggibile in chiaro.
- Identificatori e risorse stringa non sono mascherati, né è presente crittografia o packing del codice.
- La mancanza di offuscamento abbassa drasticamente la complessità per condurre attività di reverse engineering e repackaging dell'app.

Mitigazione

- Configurare strumenti di offuscamento del codice su classi, metodi e offuscare/cifrare le stringhe sensibili.

Assenza di rilevamento anti-debugging

Descrizione

- L'app non implementa alcun rilevamento anti-debugging, né a livello Java (Debug.isDebuggerConnected, waitForDebugger, flag FLAG_DEBUGGABLE) né a livello nativo (ptrace, lettura di TracerPid).
- L'assenza dell'attributo android:debuggable è solo una protezione passiva di base, non un rilevamento attivo: su dispositivo rooted o ripacchettizzando in debug l'analisi a runtime resta possibile.
- Le occorrenze del termine «Debug» nel codice corrispondono unicamente a funzioni di logging dell'in-app billing.

Mitigazione

- Introdurre più controlli anti-debugging distribuiti e su più livelli (Java e nativo), integrati con i controlli di integrità del codice.

Finding deboli

#	Vulnerabilità	Descrizione
1	Iterazioni KDF ridotte (24.000)	SQLCipher usa 24k iterazioni vs 256k dello standard attuale: abbassa il costo di un brute-force offline sulla Master Password.
2	Chiave KeyStore (PIN) non vincolata all'auth	La chiave che protegge la Master Password per lo sblocco con PIN non usa setUserAuthenticationRequired(true): utilizzabile dal processo senza autenticazione esplicita.
3	Metadati in chiaro (Keychain4)	Nel vecchio formato vault, i campi Title/Type restano in chiaro: un vault legacy può rivelare i nomi dei servizi senza Master Password.x
4	Firma RSA 1024 bit (no v3)	L'APK è firmato dal certificato di produzione, ma con chiave RSA a 1024 bit (debole, std ≥ 2048) e senza schema di firma v3.
5	Pulizia WebView incompleta	La funzione "Clear web data" è solo manuale e non invoca WebStorage.deleteAllData(): il DOM storage HTML5 non viene ripulito.
6	Permessi eccessivi/malformati	Permessi non necessari (CALL_PHONE, READ_CONTACTS...), alcuni deprecati, e una voce di permesso malformata (refuso android.permission.Android O).
7	SDK di tracking non necessari	Google Analytics e Tag Manager sono impacchettati ma inerti (mai invocati): dipendenze residue che ampliano la superficie/privacy.
8	Cifratura non autenticata	AES in modalità CBC senza authenticated encryption (GCM) né HMAC esplicito su alcune componenti: nessuna garanzia d'integrità del ciphertext.
9	Traffico in chiaro non disabilitato	usesCleartextTraffic non impostato a false (targetSdk 24): HTTP tecnicamente permesso, anche se non usato per dati sensibili.
10	PendingIntent senza FLAG_IMMUTABLE	I PendingIntent sono mutabili di default, ma incapsulano Intent espliciti verso componenti interni: scenario di dirottamento neutralizzato.

**Grazie per
l'attenzione**